

Détection des étudiants à risque (ML)

Comment la plateforme repère tôt les étudiants en difficulté — expliqué depuis ton code

Plateforme d'apprentissage adaptatif · Yassine Ben Nessib · EspritTech — expliqué à partir du vrai code source

En une phrase — On calcule **8 nombres** décrivant l'activité d'un étudiant, on les donne à un classifieur entraîné, et on obtient une **probabilité de risque** + les **facteurs principaux** derrière. Code : `risk_service.py` (usage) + `train_risk_model.py` (entraînement).

1 L'idée en une minute

Beaucoup d'étudiants galèrent en silence. Si la plateforme peut les **repérer tôt**, un enseignant peut aider avant l'abandon. C'est un système d'**alerte précoce** basé sur l'**apprentissage supervisé** : décrire chaque étudiant par des nombres (features), entraîner un modèle sur des exemples passés, prédire qui est à risque.

2 Où ça vit dans le code

`app/services/risk_service.py`

Calcule les 8 features et lance la prédiction.

`scripts/train_risk_model.py`

Entraîne et évalue le modèle.

`app/data/risk_model.joblib`

Le modèle entraîné + le scaler sauvegardés.

`scripts/generate_risk_data.py`

Crée les données d'entraînement (simulées).

3 Étape 1 — construire les 8 features depuis la base

Le modèle a besoin de nombres. `extract_features()` lit le vrai historique de quiz de l'étudiant en base et calcule 8 signaux.

RISK_SERVICE.PY — EXTRACT_FEATURES() (EXTRAIT)

```
results = db.query(QuizResult).filter(...).all()
scores = [float(r.score) for r in results]
avg_score = sum(scores) / len(scores) # 1. average score
quizzes_count = len(results) # 2. how many quizzes
avg_time_min = sum(r.temps_reponse for r in results)/len(results)/60 # 3. time
days_since_last = (now - last).total_seconds()/86400 # 4. inactivity
score_trend = np.polyfit(xs, scores, 1)[0] # 5. going up or down?
avg_mastery = sum(mvals)/len(mvals) # 6. BKT mastery (Brick 1)
low_mastery_ratio = (# concepts below 40) / total # 7. weak concepts
error_rate = wrong / total # 8. error rate
```

Ce que ça fait, étape par étape :

- Elle récupère les lignes `QuizResult` de l'étudiant en base (son historique de quiz).
- Elle calcule ensuite 8 nombres : score moyen, nombre de quiz, temps moyen, jours depuis la dernière activité, la **tendance** des scores (`np.polyfit` ajuste une droite — une pente négative = scores qui baissent), la maîtrise BKT moyenne, la part de concepts faibles, et le taux d'erreur.
- Ces 8 features sont **exactement** celles des données d'entraînement, donc un modèle entraîné sur des données simulées se branche directement sur les vraies données.

4 Étape 2 — prédire le risque

RISK_SERVICE.PY — PREDICT_FROM_FEATURES()

```
x = np.array([[float(features[f]) for f in order]]) # put the 8 numbers in order
xs = bundle["scaler"].transform(x) # standardise them
proba = float(bundle["clf"].predict_proba(xs)[0][1]) # P(at risk)
at_risk = proba >= bundle["threshold"] # flag if above 0.5
# explain WHY: coefficient x standardised value
contrib = bundle["clf"].coef_[0] * xs[0]
top_factors = [name for name, c in sorted(zip(order, contrib),
                                          key=lambda kv: -kv[1]) if c > 0][:3]
return {"at_risk": at_risk, "probability": proba, "top_factors": top_factors}
```

Ce que ça fait, étape par étape :

- Les 8 features sont mises dans le bon **ordre** et standardisées par le `scaler` sauvegardé (pour que « 80 % » et « 12 minutes » deviennent comparables).
- `predict_proba(...)[0][1]` donne la probabilité de la classe « à risque » (colonne 1).
- Si cette probabilité dépasse le seuil (0.5), l'étudiant est marqué `at_risk = True`.
- **Explication :** `coef_ × valeur` indique combien chaque feature a poussé cet étudiant vers le risque ; on remonte les **3 principaux facteurs positifs** — l'enseignant voit donc *pourquoi*.

5 Étape 3 — comment le modèle a été entraîné

TRAIN_RISK_MODEL.PY — LE PIPELINE D'ENTRAÎNEMENT

```
scaler = StandardScaler().fit(X)          # learn the scale of each feature
Xs = scaler.transform(X)
X_tr, X_te, y_tr, y_te = train_test_split(Xs, y, test_size=0.2, stratify=y)
clf = LogisticRegression(max_iter=2000, class_weight="balanced")
clf.fit(X_tr, y_tr)                      # learn
roc = roc_auc_score(y_te, clf.predict_proba(X_te)[: ,1]) # evaluate
bundle = {"scaler": scaler, "clf": clf, "features": features, "threshold": 0.50}
joblib.dump(bundle, OUT)
```

Ce que ça fait, étape par étape :

- `StandardScaler` met chaque feature à la même échelle (moyenne 0, écart 1) pour qu'aucune ne domine juste parce que ses nombres sont plus grands.
- `class_weight="balanced"` est important : en général peu d'étudiants sont à risque, donc on dit au modèle de prêter plus attention à cette classe rare.
- On évalue avec le **ROC-AUC** et un rapport ; pour une alerte précoce, c'est le **rappel (recall)** qui compte le plus (mieux vaut une fausse alerte qu'un étudiant manqué).
- Le scaler + le classifieur + l'ordre des features sont sauvegardés ensemble dans le `.joblib`, pour que le service refasse exactement le même traitement.

POURQUOI STANDARDISER (COMPARER DES UNITÉS DIFFÉRENTES ÉQUITABLEMENT)

$\text{valeur standardisée} = (\text{valeur} - \text{moyenne}) / \text{écart_type}$

6 Exemple concret

Un étudiant signalé

Score moyen faible, peu de tentatives, plusieurs quiz ratés, tendance qui baisse → le modèle sort `probability = 0.82`. Au-dessus de 0.5 → `at_risk = True`, avec `top_factors = ["low_mastery_ratio", "error_rate", "days_since_last"]`. L'enseignant reçoit un signal clair et expliqué.

7 Forces & limites

Forces

- Alerte précoce avant l'échec
- Utilise des données déjà collectées
- Explique les principaux facteurs
- La probabilité permet de régler la prudence

Limites

- Nécessite des données représentatives
- Données biaisées → modèle biaisé
- Corrélation n'est pas causalité
- Le seuil est un compromis précision/ rappel

Note honnête — Le modèle a été entraîné sur des étudiants **synthétiques**

(`generate_risk_data.py`), il démontre donc la méthode. Étapes honnêtes suivantes : entraîner

sur de **vraies** cohortes, valider, et vérifier l'**équité**. N'utiliser le signalement que comme aide, jamais pour punir.

8 Q&R

QUESTIONS DE SOUTENANCE / ENTRETIEN (DONT « CE BLOC FAIT QUOI ? »)

Q Quelles sont les 8 features et d'où viennent-elles ?

R Score moyen, nombre de quiz, temps moyen, jours depuis la dernière activité, tendance des scores, maîtrise BKT moyenne, ratio de concepts faibles, taux d'erreur — toutes calculées depuis l'historique de quiz en base par `extract_features()`.

Q Que fait `predict_from_features()` ?

R Elle ordonne et standardise les features, obtient la probabilité de risque, marque l'étudiant au-dessus du seuil, et calcule les facteurs principaux qui expliquent la décision.

Q Comment expliques-tu pourquoi un étudiant est signalé ?

R $\text{contrib} = \text{coefficient} \times \text{valeur standardisée}$ pour chaque feature ; les plus grandes contributions positives sont remontées comme facteurs principaux.

Q Pourquoi `StandardScaler` ?

R Pour comparer équitablement des features d'unités différentes (pourcent, minutes, comptes), afin que le modèle ne soit pas biaisé par l'échelle.

Q Pourquoi `class_weight=balanced` ?

R Parce que les étudiants à risque sont rares (déséquilibre des classes) ; l'équilibrage pousse le modèle à bien les attraper.

Q Comment est-ce évalué et pourquoi le rappel ?

R Avec exactitude, précision, rappel, F1 et ROC-AUC ; le rappel compte le plus car manquer un étudiant en difficulté est l'erreur coûteuse.

Q Un souci éthique ?

R Oui — éviter les biais, protéger les données, et n'utiliser le signal que pour offrir de l'aide, pas pour pénaliser.

9 Termes clés

Feature — Un nombre décrivant l'étudiant.

Étiquette (label) — Le résultat connu (à risque : 1 / 0).

StandardScaler — Met les features à la même échelle.

Régression logistique — Classifieur simple et explicable donnant des probabilités.

class_weight balanced — Se concentrer sur la classe rare.

predict_proba — Probabilité de chaque classe.

ROC-AUC — À quel point le modèle sépare les deux classes.

Rappel (recall) — Part des vrais étudiants à risque qu'on attrape.